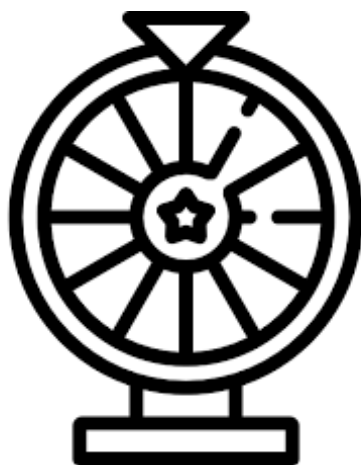


Licenciatura em Engenharia Informática e de Computadores



JOGO DA ROLETA
(*Roulette Game*)

Projeto
de
Laboratório de Informática e Computadores
2024 / 2025 verão

Duarte Rodrigues, Alexandre Silva, Daniel Pereira

04 de junho de 2025

Conteúdo

1	Descrição	1
2	Arquitetura do Sistema	3
2.1	<i>Keyboard Reader</i>	3
2.2	<i>Coin Acceptor</i>	3
2.3	<i>Serial LCD Controller</i>	4
2.4	<i>Serial Roulette Controller</i>	4
2.5	<i>Control</i>	4
3	Resultados Experimentais	6
4	Conclusões	7
5	Trabalho Futuro	8
	Referências	9
A	Roulette Game - Hardware	10
A.1	RouletteGame.vhd	11
A.2	Atribuição de Pinos	14
B	Roulette Game - Software (<i>Control</i>)	15
B.1	Roulette Game - App	15
B.2	Roulette Display	17
B.3	M	19
B.4	TUI	20
B.5	Coin Acceptor	20
B.6	Coin Deposit	21
B.7	Statistics	21
B.8	RouletteDisplay	21
B.9	File Access	22

Lista de Figuras

1	Diagrama de blocos do jogo da Roleta (<i>Roulette Game</i>)	1
2	Arquitetura do sistema que implementa o jogo da Roleta (<i>Roulette Game</i>) . . .	3
3	Bloco <i>Coin Acceptor</i>	4
a	Diagrama de blocos	4
b	Diagrama temporal	4
4	Diagrama de blocos do módulo <i>Serial LCD Controller (SLCDC)</i>	4
5	Diagrama de blocos do módulo <i>Serial Roulette Controller (SRC)</i>	4
6	Diagrama lógico do Jogo da Roleta (<i>Roulette Game</i>)	5

Lista de Tabelas

1	Tabela de atribuição de pinos do projeto (.qsf)	14
---	--	----

Lista de Algoritmos

1	Código VHDL - Arquitetura Structural do sistema	11
---	---	----

1 Descrição

Este projeto implementa o jogo da Roleta (*Roulette Game*), no qual a roleta compreende números entre 0 e 9, e alfabetos entre A e D, um jogador realiza apostas premindo as teclas de um teclado correspondentes aos números/alfabetos em que pretende apostar. Por cada aposta é debitado um crédito ao saldo acumulado do jogador, podendo o jogador apostar mais do que um crédito num mesmo número. Os créditos são obtidos pela introdução de moedas no moedeiro, este só aceita dois tipos de moeda, que correspondem a dois (2) e quatro (4) créditos.

O sistema que implementa o jogo é constituído por: um PC (*Control*); um teclado de 16 teclas; um moedeiro (*Coin Acceptor*); um display *Liquid Cristal Display (LCD)* de duas linhas com 16 caracteres; um display da roleta (*Roulette Display*) e uma chave de manutenção designada por *M*, para colocação do sistema em modo de Manutenção. Na Figura 1 apresenta-se o diagrama de blocos do jogo da Roleta.

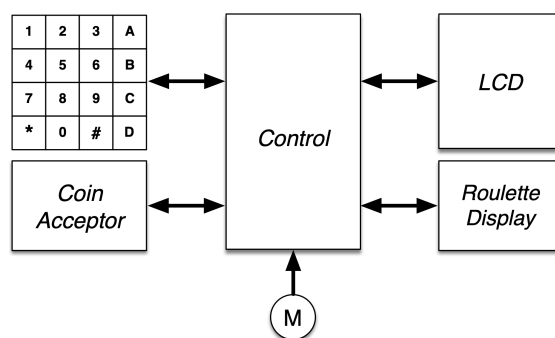


Figura 1: Diagrama de blocos do jogo da Roleta (*Roulette Game*)

Sobre o sistema proposto podem realizar-se as seguintes ações em modo de Jogo:

- **Jogo** – Inicia-se o jogo premindo a tecla '*', e existam créditos disponíveis. Utilizando as teclas numéricas (0-9) e alfabéticas (A-D) realizam-se as apostas, retirando-se um crédito ao saldo do jogador por cada aposta realizada. O jogador termina as apostas premindo a tecla '#', o que dá início ao sorteio. Durante um tempo aleatório, o sistema simula o girar da Roleta no *Roulette Display*, permitindo ainda realizar apostas até 5 segundos antes desta parar. Ao parar a Roleta, o número sorteado e os créditos obtidos na jogada são apresentados no *Roulette Display*. Os créditos obtidos são acumulados após 5 segundos ao saldo do jogador, também apresentados no *LCD*.

No modo Manutenção podem realizar-se as seguintes ações sobre o sistema:

- **Teste** – Premindo a tecla '*' inicia-se um jogo sem créditos e sem contabilizar os números sorteados.
- **Consultar os contadores de moedas e jogos** – Carregando na tecla 'A' permite-se a listagem dos contadores de moedas e jogos realizados.
- **Iniciar os contadores de moedas e jogos** – Premindo a tecla 'A' e em seguida a tecla '*', o sistema de gestão coloca os contadores de moedas e jogos a zero, iniciando um novo ciclo de contagem.

- **Consultar a lista de números sorteados** – Carregando na tecla ‘C’ permite-se a listagem dos números sorteados.
- **Iniciar a lista de números sorteados** – Premindo a tecla ‘C’ e em seguida a tecla ‘*’, o sistema inicia um novo ciclo de estatística de números sorteados.
- **Desligar** – Premindo a tecla ‘D’ permite desligar o sistema, que encerra apenas após a confirmação do utilizador, ou seja, o programa termina e as estruturas de dados, contendo a informação dos contadores e da Lista de Números Sorteados, são armazenadas de forma persistente em dois ficheiros de texto, por linha e com os campos de dados separados por “;”. O primeiro ficheiro deverá conter o número de jogos realizados e o número de moedas guardadas no cofre do moedeiro. O segundo ficheiro deverá conter a Lista de Números Sorteados, que contém o número de saídas e os prémios atribuídos por cada número. Os dois ficheiros devem ser carregados para o sistema no seu processo de arranque.

2 Arquitetura do Sistema

O sistema é implementado numa solução híbrida de hardware e software, como apresentado no diagrama de blocos da Figura 2. A arquitetura proposta é constituída por cinco módulos principais: *i*) um leitor de teclado, designado por *Keyboard Reader*; *ii*) um módulo de interface com o *LCD*, designado por *Serial LCD Controller (SLCDC)*; *iii*) um módulo de interface com o display da roleta (*Roulette Display*), designado por *Serial Roulette Controller (SRC)*; *iv*) um moedeiro, designado por *Coin Acceptor*; e *v*) um módulo de controlo, designado por *Control*. Os módulos *i*), *ii*) e *iii*) deverão ser implementados em hardware, o moedeiro (*iv*) será simulado utilizando um interruptor e um *LED*, enquanto o módulo de controlo (*v*) deverá ser implementado em software, descrito em linguagem *Kotlin* e executado num PC.

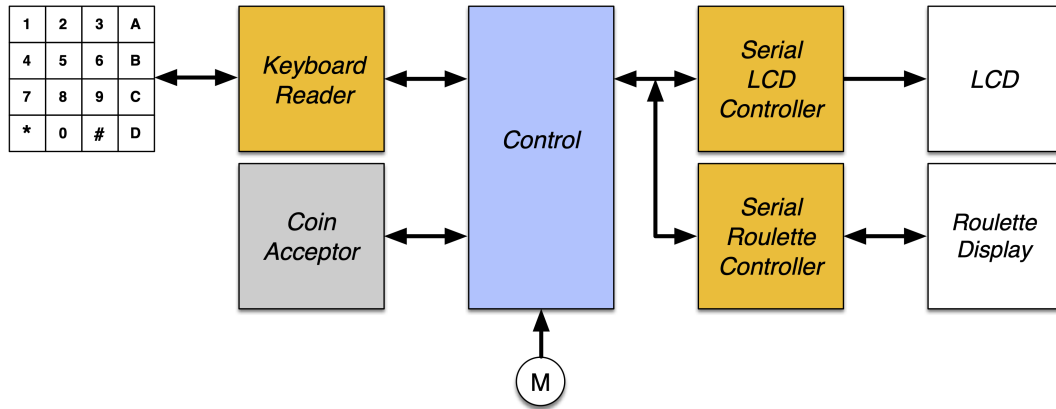


Figura 2: Arquitetura do sistema que implementa o jogo da Roleta (*Roulette Game*)

O módulo *Control* processa os dados e envia a informação a apresentar no *LCD* através do módulo *SLCDC*. O *Roulette Display* é atuado pelo módulo *Control*, através do módulo *SRC*. Ppor forma a minimizar o número de interligações, a comunicação entre o módulo *Control* e os módulos *SLCDC* e *SRC* é realizada através de um protocolo série.

2.1 Keyboard Reader

O módulo *Keyboard Reader*, descrito em [1] é responsável pela descodificação do teclado matricial de 16 teclas, determinando qual a tecla pressionada e disponibilizando o seu código, com quatro bits, ao módulo *Control*. Caso este não esteja disponível para o receber imediatamente, o código da tecla é armazenado internamente, até ao limite de dezoito (18) códigos.

2.2 Coin Acceptor

O módulo *Coin Acceptor* implementa a interface com o moedeiro, sinalizando ao módulo *Control* que o moedeiro recebeu uma moeda através da ativação do sinal *Coin*, identificando o tipo de moeda pelo sinal *Coin_{id}*. A entidade consumidora informa o *Coin Acceptor* que já contabilizou a moeda ativando o sinal *accept*, conforme apresentado no diagrama temporal da Figura 3b.

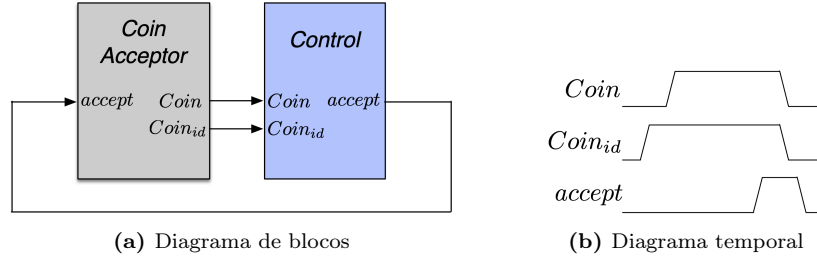


Figura 3: Bloco *Coin Acceptor*

2.3 Serial LCD Controller

O módulo *Serial LCD Controller* (*SLCDC*) implementa a interface com o *LCD*, fazendo a receção em série da informação enviada pelo módulo de controlo e entregando-a posteriormente ao *LCD*, conforme representado na Figura 4, e descrito em [2].

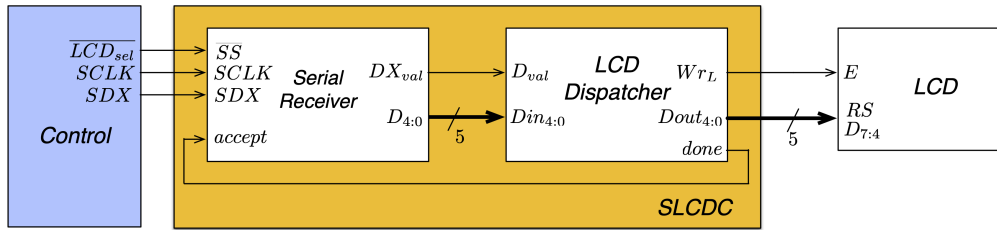


Figura 4: Diagrama de blocos do módulo *Serial LCD Controller* (*SLCDC*)

2.4 Serial Roulette Controller

O módulo *Serial Roulette Controller* (*SRC*) implementa a interface com o display da roleta (*Roulette Display*), realizando a receção em série da informação enviada pelo módulo de controlo e entregando-a posteriormente ao display de pontuação, conforme representado na Figura 5, e descrito em [3].

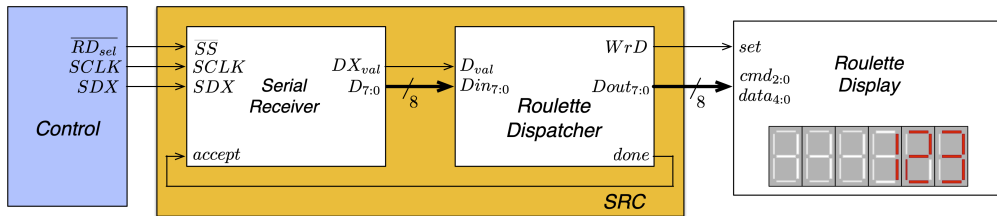


Figura 5: Diagrama de blocos do módulo *Serial Roulette Controller* (*SRC*)

2.5 Control

A implementação do módulo *Control* é realizada em software, usando a linguagem *Kotlin* e seguindo a arquitetura lógica apresentada na Figura 6.

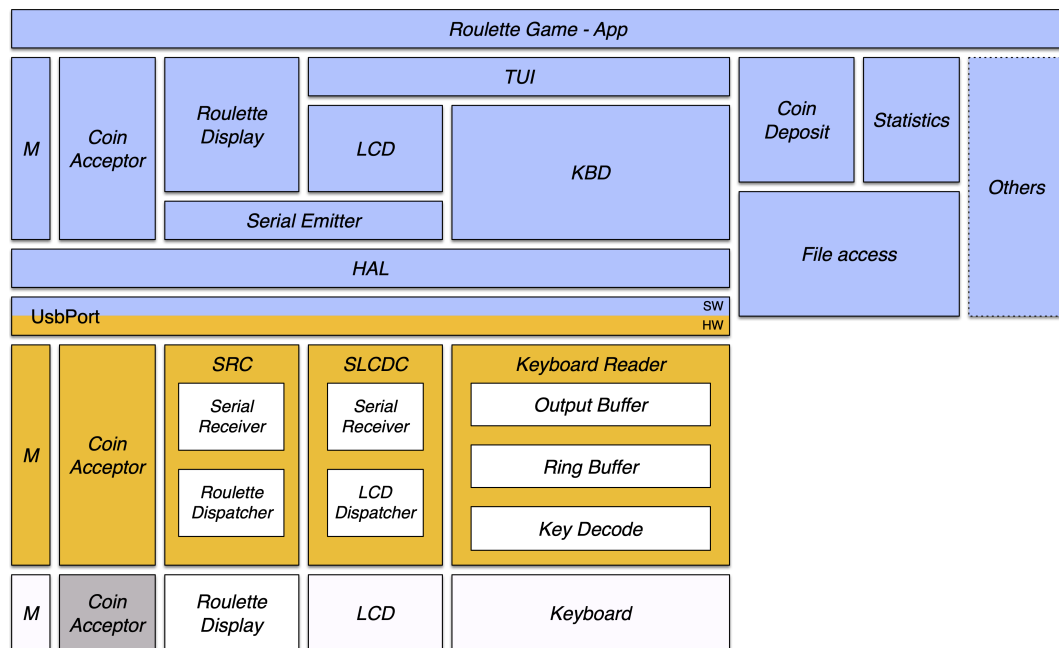


Figura 6: Diagrama lógico do Jogo da Roleta (*Roulette Game*)

3 Resultados Experimentais

Para verificar o correto funcionamento do sistema *Jogo da Roleta*, foram realizados testes modulares e integrados sobre os diversos componentes de hardware e software. Estes testes podem ser divididos em três fases:

1. Testes individuais dos módulos hardware:

- O módulo *Keyboard Reader* foi testado utilizando a introdução manual de teclas. Verificou-se a correta detecção, armazenamento em FIFO (Ring Buffer) e entrega ao módulo de controlo, com até 18 teclas armazenadas.
- O *Coin Acceptor* foi validado com dois interruptores simulando as moedas de 2 e 4 créditos. Confirmou-se que os sinais *Coin*, *Coinid* e *accept* foram corretamente gerados e interpretados.
- Os módulos *SLCDC* e *SRC* foram testados com tramas enviadas pelo software de controlo. Foi verificada a receção série, paridade ímpar e ativação dos sinais *WrL* e *done*, com apresentação correta no LCD e no display da roleta.

2. Testes do módulo de controlo (software):

- Implementado em *Kotlin*, o módulo de controlo foi testado em ambiente de simulação com os sinais I/O. A lógica de jogo, estados do menu, contadores e estatísticas funcionaram conforme o esperado.
- Foram testadas todas as funcionalidades principais: início de jogo, apostas, sorteio, atribuição de prémios e visualização dos resultados.

3. Testes do sistema completo na plataforma DE10-Lite:

- Após a integração dos módulos, o sistema foi testado na placa FPGA. Verificou-se o funcionamento completo desde a introdução de moedas até à visualização do número sorteado e atribuição dos créditos.
- O modo de manutenção foi testado com sucesso, incluindo a consulta e reinício de contadores e estatísticas.

4 Conclusões

O projeto *Jogo da Roleta* permitiu conceber, implementar e testar um sistema completo, baseado numa arquitetura híbrida de hardware e software, que simula de forma realista a dinâmica de um jogo interativo de apostas. Através da utilização da linguagem *Kotlin* para o desenvolvimento da lógica de controlo e da implementação de módulos de hardware em VHDL, os alunos tiveram a oportunidade de aplicar e consolidar conhecimentos adquiridos nas áreas de sistemas digitais, protocolos de comunicação série e as suas ligações com o hardware da placa DE10-Lite.

A integração dos vários módulos — *Keyboard Reader*, *SLCDC*, *SRC* e o módulo *Control* — demonstrou ser funcional e coerente, tanto do ponto de vista técnico como didático. Cada componente foi desenvolvido com uma interface clara e objetivos bem definidos, o que facilitou a sua compreensão individual e a posterior integração no sistema global.

Durante o desenvolvimento, destacou-se a importância de garantir uma comunicação fiável entre os componentes, especialmente no que respeita à transmissão série de dados com verificação de paridade e sincronização de sinais de controlo. A simulação da interface com o utilizador, através do LCD e do display da roleta, revelou-se essencial para garantir uma experiência de utilização fluída e compreensível.

Do ponto de vista funcional, os testes experimentais demonstraram que o sistema cumpre todos os requisitos definidos no enunciado do projeto. Foram validadas as funcionalidades em modo de jogo (apostas, sorteio, contagem e atribuição de créditos) e em modo de manutenção (consulta e reinicialização de contadores, histórico de números sorteados).

Este projeto evidenciou a complexidade e os desafios associados ao desenvolvimento deste tipo de sistemas e demonstrou a eficácia da divisão modular para promover a escalabilidade, reutilização de componentes e facilidade de manutenção. A utilização da placa DE10-Lite permitiu validar fisicamente o comportamento dos módulos desenvolvidos, reforçando a ligação entre teoria e prática.

Em suma, o trabalho permitiu alcançar com sucesso os objetivos propostos.

5 Trabalho Futuro

Futuros melhoramentos para o projeto podem incluir:

- **Aprimoramento da animação da roleta:** Implementar uma animação mais fluida e com velocidade variável no display da roleta ou uma lista de animações possíveis e aleatórias, criando uma experiência diferente a cada aposta.
- **Adição de novos modos de jogo:** Divisão de apostas entre números pares/ímpares e um jackpot, simbolizando as cores das roletas (vermelho/preto e verde).
- **Melhorias na interface de manutenção:** Com visualização mais detalhada de estatísticas e histórico dos jogos.
- **Criação de perfis e histórico de jogadores:** Criar a possibilidade de um ambiente competitivo entre diversos utilizadores, registando perfis e valores que estes obteram, com a possibilidade de criar um ranking com os melhores resultados.
- **Internacionalização da interface:** Adicionar suporte a múltiplos idiomas (Português/Inglês) no LCD e no TUI da aplicação, tornando o jogo acessível a um público mais vasto.
- **Implementação de som e feedback:** Integrar um sistema de som simples para dar feedback ao utilizador durante ações como introdução de moeda, início do jogo, apostas válidas ou vitória.

Estas melhorias tornariam o projeto em algo mais complexo, mas mantendo a estrutura e o conceito desenvolvido, tornando possível uma aplicação real de entretenimento digital e uma melhor experiência para os utilizadores.

Referências

- [1] David Velez, Nuno Sebastião, Pedro Miguens Matutino, Rogério Rebelo, Rui Duarte *KeyBoardReader datasheet*, 2025, ISEL.
- [2] David Velez, Nuno Sebastião, Pedro Miguens Matutino, Rogério Rebelo, Rui Duarte *Serial LCD Controller datasheet*, 2025, ISEL.
- [3] David Velez, Nuno Sebastião, Pedro Miguens Matutino, Rogério Rebelo, Rui Duarte *Serial Roulette Display Controller datasheet*, 2025, ISEL.
- [4] Terasic *DE10-lite User Manual*, 2020, Terasic.
- [5] Terasic *de10-Lite Exp. Board - Reference Manual*, 2022, Pedro Miguens Matutino

A Roulette Game - Hardware

A implementação em hardware do *Roulette Game* foi realizada com recurso aos blocos modulares anteriormente referidos, promovendo a reutilização, a clareza funcional e a facilidade de teste e integração.

Os principais módulos desenvolvidos em hardware foram:

- **Keyboard Reader:** responsável pela varrimento e descodificação do teclado matricial 4x4, armazenando os códigos das teclas premidas e entregando-os ao módulo de controlo. Este bloco inclui um buffer circular (FIFO) e um mecanismo de controlo de fluxo com os sinais *Dval*, *Kack*, *CTS* e *ACK*.
- **Serial LCD Controller (SLCDC):** recebe, em série, tramas de 5 bits de dados mais 1 bit de paridade, provenientes do software de controlo. Após validação da trama, entrega os dados ao LCD através dos sinais apropriados e ativa o sinal *WrL*.
- **Serial Roulette Controller (SRC):** funciona de forma semelhante ao SLCDC, mas processa tramas de 8 bits de dados e 1 bit de paridade, correspondentes aos comandos e valores a exibir no display da roleta.
- **USBPort:** O meio de ligação entre o hardware e o software, que permite que a comunicação dos dados do sistema e os valores definidos em cada componente cheguem ao local pretendido.
- **Coin Acceptor - 2 Switches:** simula a entrada de moedas no sistema, ativando os sinais *Coin* e *Coinid*, de acordo com o tipo de moeda inserida (2 ou 4 créditos).
- **M - Switch:** simula a entrada do modo de manutenção do sistema, enviando o sinal para o USBPort que por sua vez comunica com o Control.
- **RoulettDisplay - 6 Displays de 7seg:** a entidade com todos os displays onde vai decorrer o jogo, animação e apostas.

Todos os módulos foram desenvolvidos para funcionar de forma síncrona, respeitando os protocolos de comunicação definidos, e foram testados individualmente antes da integração no sistema global. A comunicação com os módulos SLCDC e SRC foi realizada através de um emissor série, implementado em software, que transmite os dados com sincronização através do sinal *SCLK* e do bit de paridade ímpar para validação.

A arquitetura modular e a utilização de comunicação série permitiram reduzir o número de sinais físicos entre o PC e a FPGA. A validação final foi realizada na placa DE10-Lite, com sucesso, comprovando o funcionamento coordenado de todos os componentes.

O ficheiro `RouletteGame.vhd` corresponde à arquitetura estrutural do sistema implementado em hardware. Este ficheiro realiza a interligação entre os diversos módulos desenvolvidos.

Segue-se o código VHDL completo do componente:

A.1 RouletteGame.vhd

Algoritmo 1: Código VHDL - Arquitetura Structural do sistema

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity roulette is
  Port (
    clk      : in  std_logic;
    reset    : in  std_logic;
    Coin     : in  std_logic;
    Coin_id  : in  std_logic;
    Lines    : in  std_logic_vector(3 downto 0);
    M        : in  std_logic;

    KEY_COL  : out std_logic_vector(3 downto 0);
    LCD_DATA : out std_logic_vector(7 downto 4);
    LCD_RS   : out std_logic;
    LCD_EN   : out std_logic;
    accept   : out std_logic;
    HEX0, HEX1,
    HEX2, HEX3,
    HEX4, HEX5 : out std_logic_vector(7 downto 0)
  );
end roulette;

architecture Structural of roulette is

  -- Declara o dos componentes
  component keyboard_reader ...
  end component;

  component UsbPort ...
  end component;

  component SRC ...
  end component;

  component SLCDC ...
  end component;

  component rouletteDisplay ...
  end component;

  -- Sinais internos
  signal sig_Q      : std_logic_vector(3 downto 0);
  signal sig_Dval   : std_logic;
  signal ACK        : std_logic;

  signal sig_inputPort : std_logic_vector(7 downto 0);
  signal sig_outputPort : std_logic_vector(7 downto 0);

  signal Dout_SRC      : std_logic_vector(7 downto 0);
```

```
signal sig_WRD          : std_logic;

signal Dout_LCD         : std_logic_vector(4 downto 0);
signal sig_WRL          : std_logic;

signal sig_cmd          : std_logic_vector(2 downto 0);
signal sig_data         : std_logic_vector(4 downto 0);
signal sig_set          : std_logic;

signal internal_SDx     : std_logic;
signal internal_LCDSel  : std_logic;
signal internal_RouSel  : std_logic;
signal internal_SCLK    : std_logic;

begin

  -- Instancia do teclado
  U5: keyboard_reader
    port map (
      lines      => Lines,
      CLK        => clk,
      Reset      => reset,
      ack_control => ACK,
      columns    => KEY_COL,
      output     => sig_Q,
      Dval       => sig_Dval
    );

  -- Instancia da USBPort
  U1: UsbPort
    port map (
      inputPort  => sig_inputPort,
      outputPort => sig_outputPort
    );

  -- Instancia do receptor serial (SRC)
  U2: SRC
    port map (
      RouSel => internal_RouSel,
      SCLK   => internal_SCLK,
      SDx    => internal_SDx,
      MClk   => clk,
      Reset  => reset,
      Wrl    => sig_WRL,
      Dout   => Dout_SRC
    );

  -- Instancia do SLCDC
  U4: SLCDC
    port map (
      LCDSel => internal_LCDSel,
      SCLK   => internal_SCLK,
      SDx    => internal_SDx,
      MClk   => clk,
```

```
Reset    => reset,
Wrl      => sig_WRL,
Dout     => Dout_LCD
);

-- Ligacao da saida do SRC aos sinais de comando e dados
sig_cmd  <= Dout_SRC(2 downto 0);
sig_data <= Dout_SRC(7 downto 3);
sig_set  <= sig_WRD;

-- Instancia do display da roleta
U3: rouletteDisplay
    port map (
        cmd    => sig_cmd,
        data   => sig_data,
        set    => sig_set,
        HEX0   => HEX0,
        HEX1   => HEX1,
        HEX2   => HEX2,
        HEX3   => HEX3,
        HEX4   => HEX4,
        HEX5   => HEX5
    );

-- Atribuicoes internas
sig_inputPort(0)    <= sig_Dval;
sig_inputPort(4 downto 1) <= sig_Q;
sig_inputPort(5)    <= Coin_id;
sig_inputPort(6)    <= Coin;
sig_inputPort(7)    <= M;

internal_SDx    <= sig_outputPort(1);
accept         <= sig_outputPort(2);
ACK            <= sig_outputPort(4);
internal_LCDSel <= sig_outputPort(5);
internal_RouSel <= sig_outputPort(6);
internal_SCLK   <= sig_outputPort(7);

LCD_DATA <= Dout_LCD(4 downto 1);
LCD_RS   <= Dout_LCD(0);
LCD_EN   <= sig_WRL;

end Structural;
```

A.2 Atribuição de Pinos

Sinal	Pino
CLK	
clk	P11
LCD	
LCD_DATA[4]	W11
LCD_DATA[5]	AA10
LCD_DATA[6]	Y8
LCD_DATA[7]	Y7
LCD_RS	W8
LCD_EN	V5
Display HEX0	
HEX0[0]	C14
HEX0[1]	E15
HEX0[2]	C15
HEX0[3]	C16
HEX0[4]	E16
HEX0[5]	D17
HEX0[6]	C17
HEX0[7]	D15
Display HEX2	
HEX2[0]	B20
HEX2[1]	A20
HEX2[2]	B19
HEX2[3]	A21
HEX2[4]	B21
HEX2[5]	C22
HEX2[6]	B22
HEX2[7]	A19
Display HEX4	
HEX4[0]	F18
HEX4[1]	E20
HEX4[2]	E19
HEX4[3]	J18
HEX4[4]	H19
HEX4[5]	F19
HEX4[6]	F20
HEX4[7]	F17

Sinal	Pino
Display HEX1	
HEX1[0]	C18
HEX1[1]	D18
HEX1[2]	E18
HEX1[3]	B16
HEX1[4]	A17
HEX1[5]	A18
HEX1[6]	B17
HEX1[7]	A16
Display HEX3	
HEX3[0]	F21
HEX3[1]	E22
HEX3[2]	E21
HEX3[3]	C19
HEX3[4]	C20
HEX3[5]	D19
HEX3[6]	E17
HEX3[7]	D22
Display HEX5	
HEX5[0]	J20
HEX5[1]	K20
HEX5[2]	L18
HEX5[3]	N18
HEX5[4]	M20
HEX5[5]	N19
HEX5[6]	N20
HEX5[7]	L19
Entradas Digitais / Gerais	
Coin	C11
Coin_id	D12
M	C12
reset	C10
Linhas	
Lines[0]	W5
Lines[1]	AA14
Lines[2]	W12
Lines[3]	AB12
Teclado (KEY_COL)	
KEY_COL[0]	AB11
KEY_COL[1]	AB10
KEY_COL[2]	AA9
KEY_COL[3]	AA8

Tabela 1: Tabela de atribuição de pinos do projeto (.qsf)

B Roulette Game - Software (*Control*)

B.1 Roulette Game - App

O módulo *App* é o ponto de entrada e de execução principal do programa. Este foi implementado seguindo a arquitetura na Descrição (1).

Durante a execução do programa, o utilizador apostará em um ou vários números, e caso acerte em algum, ganhará 4x os créditos dispendidos.

Para controlar as fases do programa foi implementada uma máquina de estados e um *enum* de modos de jogo. Esta máquina de estados, provém-nos a cada estado, um método diferente para chamar, realizando as funções do jogo. Abaixo estão os métodos implementados:

`stepPhase()`

Atualiza a fase atual do jogo para a próxima fase, mostrando a mensagem correspondente no display e executando o método associado a essa fase.

`loadStatistics()`

Carrega as estatísticas do jogo a partir dos ficheiros de armazenamento persistente.

Comportamento:

- Lê o número de jogos jogados e moedas inseridas do ficheiro `stats.txt`.
- Lê o histórico dos últimos resultados de roleta do ficheiro `rolls.txt`.

`saveStatistics()`

Guarda as estatísticas atuais do jogo e o histórico de resultados nos respetivos ficheiros.

Ações:

- Escreve o número total de jogos jogados e moedas no ficheiro `stats.txt`.
- Atualiza o ficheiro `rolls.txt` com os últimos resultados da roleta.

`processCoins(timeout: Int)`

Espera pela inserção de moedas pelo utilizador durante um intervalo especificado em milissegundos.

Retorna: Nenhum valor; atualiza o estado interno de créditos e moedas.

Funcionamento:

- Aguarda até que uma moeda seja inserida (tempo máximo definido por `timeout`).
- Caso uma moeda seja inserida, incrementa o número de créditos e moedas.
- Atualiza o display com a informação dos créditos.
- Salva as estatísticas atualizadas e limpa o sinal de aceitação da moeda.

`resetBetSelection()`

Reseta a seleção de aposta visível no display.

Comportamento:

- Se estiver na fase de apostas (BETTING), limpa o valor no display.
- Caso contrário, exibe uma mensagem indicando o fim das apostas e a seleção atual.

`processBet(key: Char)`

Processa a aposta efetuada pelo utilizador, permitindo adicionar, aceitar ou limpar apostas.

Regras:

- 'C' cancela a aposta atual.
- 'A' aceita a aposta se for válida e atualiza créditos e lista de apostas.
- Apenas números são concatenados ao buffer da aposta, até ao limite máximo permitido.
- O display é atualizado conforme a fase do jogo.

`waitForStartOrMaintenance()`

Espera pelo início do jogo ou pela ativação do modo manutenção.

Fluxo:

- Mostra o valor "000000" no display.
- Aguarda o pressionar da tecla para iniciar ou entrar em manutenção.
- Se a condição de manutenção for ativada, muda o modo de operação para MAINTENANCE.
- Liga o display para iniciar o jogo.

`waitForBetOrCoins()`

Permite ao utilizador apostar ou inserir moedas até que o jogo avance para a fase seguinte.

Regras:

- Processa entradas do teclado e moedas inseridas.
- O jogo só avança se houver pelo menos uma aposta registada.
- Se não houver créditos, exibe mensagem e espera por inserção.
- Termina quando o utilizador pressiona '' para começar o giro.

`spinRoulette()`

Simula o giro da roleta, anima o display e calcula os resultados das apostas.

Funcionamento:

- Gira a roleta durante um tempo aleatório entre 10 a 15 segundos.
- Aceita apostas até faltarem 5 segundos para acabar a animação.
- Calcula o resultado final e atualiza créditos e histórico.
- Mostra o resultado no display e mensagem de ganhos/perdas.

B.2 Roulette Display

O módulo *Roulette Display* é utilizado para escrever dados nos *displays* de 7 segmentos. Para concretizar os objetivos da animação do display, foi também criado o *DisplaySequenceFactory*, que implementa uma sequência de animação e nos fornece sempre a próxima fase via *next()*. Este recurso foi utilizado no método *RouletteDisplay.animation()* em conjunção com lógica no módulo *App*. Abaixo encontram-se os métodos utilizados:

`setValue(value: String)`

Mostra um valor numérico (máx. 3 dígitos) no display de 7 segmentos.

Parâmetros:

- **value:** valor numérico em formato string (ex: "123").

Funcionamento:

- Codifica cada dígito com a respetiva posição;
- Envia os dados codificados para o display;
- Atualiza o display.

`setValue(value: Int)`

Versão sobrecarregada de `setValue`, que aceita um número inteiro.

Comportamento:

- Converte o inteiro para string e invoca `setValue(String)`.

`animation()`

Executa uma animação LED no display, simulando uma rotação.

Funcionamento:

- Seleciona a próxima sequência de segmentos via `DisplaySequenceFactory`;
- Aplica a sequência a cada posição do display;
- Atualiza o display.

`on()`

Liga o display de 7 segmentos.

Chama internamente `off(false)`.

`off()`

Desliga o display de 7 segmentos.

Chama internamente `off(true)`.

`off(enable: Boolean)`

Liga ou desliga o display conforme o valor do parâmetro.

Parâmetros:

- `enable`: se `true`, desliga o display; se `false`, liga-o.

Ações:

- Envia o comando apropriado (`CMD_OFF` ou `CMD_ON`) via `SerialEmitter`;
- Atualiza o display.

B.3 M

O módulo M representa o modo de *Manutenção* do jogo da Roleta. Este modo é ativado pelo *switch* de manutenção, permitindo executar operações administrativas sem afetar os dados do jogador atual.

As funcionalidades disponibilizadas neste modo incluem:

- Iniciar um jogo em modo de teste (sem perda de créditos ou registo de estatísticas).
- Consultar os contadores de moedas introduzidas e jogos realizados.
- Reiniciar os contadores de moedas e jogos.
- Consultar a lista de números sorteados.
- Reiniciar a lista de estatísticas dos números sorteados.
- Terminar o programa, gravando de forma persistente os dados em ficheiros e desligando o *display*.

Seguem-se as implementações dos métodos deste módulo:

`isMaintenance(): Boolean`

Verifica se o sistema está em modo de manutenção, consultando o bit correspondente no modo de operação atual através do HAL.

Retorna `true` se o modo for `MAINTENANCE`.

`waitForMaintenanceInput(): Unit`

Entra num ciclo de espera por interações do utilizador durante o modo de manutenção.

Comportamento das teclas:

- 'A' — Mostra o número de jogos realizados e créditos inseridos.
- '*' após 'A' — Reinicia os contadores de jogos e créditos.
- 'B' — Mostra o histórico das últimas jogadas (até 10 valores, 5 por linha).
- '*' após 'B' — Limpa o histórico de jogadas.
- 'D' — Mostra mensagem de confirmação de desligar.
- 'D' novamente — Guarda estatísticas, desliga o ecrã e termina a execução.

B.4 TUI

O módulo TUI (*Text-based User Interface*) é responsável pela interface com o utilizador no terminal, permitindo a interação textual com o sistema.

A TUI utiliza as funções de escrita no LCD e KBD para refletir parte da informação graficamente e gere o fluxo de entrada de teclas, com tempos de espera e validação de comandos. Seguem-se os métodos implementados.

`writeMessage(msg: String)`

Escreve uma mensagem no LCD. Primeiro limpa o display, move o cursor para a posição inicial, e escreve o conteúdo da string. Aguarda um pequeno intervalo de tempo antes da escrita (definido por `WAIT_TIME`).

`getKey(timeout: Long)`

Lê uma tecla do teclado com um timeout opcional (por defeito, 1000 ms). Retorna o caractere pressionado ou um valor especial se o tempo expirar.

B.5 Coin Acceptor

Este módulo simula o funcionamento do moedeiro e é responsável pela deteção da introdução de moedas. Dois tipos de moedas são suportados: uma atribui dois créditos e outra atribui quatro. A interface consiste em dois sinais digitais:

- **Coin**: indica que foi inserida uma moeda.
- **Coinid**: identifica o tipo de moeda (2 ou 4 créditos).

O software verifica continuamente o estado do moedeiro e atualiza os créditos do jogador em conformidade. Após contabilização, o sinal `accept` é ativado para informar o módulo físico que o crédito foi processado. Segue-se o método implementado:

`waitCoin(timeout: Int): Int`

Aguarda a deteção de uma moeda dentro do tempo limite especificado (em milissegundos).

Retorna:

- 4, se a moeda for identificada com o sinal `COIN_ID` ativo;
- 2, se a moeda for detetada mas `COIN_ID` estiver inativo;
- 0, se o tempo limite for atingido sem deteção.

Quando uma moeda é detetada, o sinal `ACCEPT` é ativado.

Para simplificar a leitura e com fim de dar mais abstracção aos sinais do *Coin Acceptor*, criou-se o *CoinSignal*, que regista os sinais relativos ao módulo, e que contém métodos a interagir com o *HAL* para alterar e ler os valores dos mesmos.

```
enum class CoinSignal
```

Enumeração que representa os sinais digitais envolvidos na deteção e validação de moedas. Cada valor enum tem uma máscara associada (bit de entrada ou saída) e um estado interno `value`.

- `COIN` — Bit de entrada (`UsbPort.I6`). Ativado quando uma moeda é inserida.
- `COIN_ID` — Bit de entrada (`UsbPort.I5`). Indica se a moeda inserida é identificável (ex: moeda válida).
- `ACCEPT` — Bit de saída (`UsbPort.O2`). Ativa-se para sinalizar que a moeda foi aceite.

Funções associadas:

- `isActive()` — Verifica se o sinal está ativo (bit definido no `HAL`).
- `set()` — Ativa o sinal, definindo o bit correspondente.
- `unset()` — Desativa o sinal, limpando o bit correspondente.

B.6 Coin Deposit

O módulo `Coin Deposit` mantém o registo acumulado de moedas inseridas, separadamente do número de créditos disponíveis. Isto é importante para fins de auditoria no modo de manutenção. A lógica deste modo encontra-se encapsulada no módulo `App`, da mesma maneira que o `Statistics`, na forma de variáveis e acessos ao `FileAccess`.

B.7 Statistics

O módulo `Statistics` é responsável por manter o histórico das jogadas.

Estes dados são utilizados para fins de monitorização e podem ser consultados no modo de manutenção. Também permitem ao administrador detetar padrões, tendências ou potenciais falhas de aleatoriedade no sorteio.

A lógica deste módulo está encapsulada no módulo `App`, na forma de uma variável `rollHistory` e acessos ao `FileAccess`.

B.8 RouletteDisplay

O módulo `RouletteDisplay` é responsável por controlar o display da roleta, através de tramas enviadas ao componente `SRG`.

Comandos possíveis:

- `init()` — Inicialização do `Serial Emitter` e ligação dos displays;
- `setValue(value: Int)` e `setValue(value: String)` — apresenta um valor, escrito em string ou inteiro;
- `animation()` — inicia a animação do sorteio;
- `off(enabled: Boolean)` — ativa ou desativa o display.

- *on()* e *off()* — chama a função anterior, com o valor booleano predefinido de acordo com o nome;

As tramas seguem o formato de 8 bits (3 de comando, 5 de dados).

B.9 File Access

O módulo *File Access* é responsável pela persistência de dados do sistema entre execuções. São utilizados dois ficheiros de texto:

- *stats.txt* — contém o número total de jogos realizados e o número de moedas introduzidas.
- *rolls.txt* — contém a lista de números sorteados, o número de vezes que cada um saiu, e os créditos atribuídos.

Os ficheiros são lidos na fase de inicialização da aplicação, e atualizados na sua terminação. O formato dos ficheiros é simples (valores separados por ponto e vírgula), garantindo compatibilidade com editores de texto e ferramentas externas.

Este módulo garante que os dados persistentes são sincronizados com os módulos *Coin Deposit* e *Statistics*, respeitando a integridade dos dados.